

Privacy-aware White and Black List Searching for Fraud Analysis

William J Buchanan¹, Jamie Gilchrist², Zakwan Jaroucheh², Dmitri Timosenko², Nanik Ramchandani², Hisham Ali¹

¹ Blockpass ID Lab, Edinburgh Napier University, Edinburgh.

² LastingAsset, Edinburgh Napier University, Edinburgh

Abstract. In many areas of cybersecurity, we require access to Personally Identifiable Information (PII), such as names, postal addresses and email addresses. Unfortunately, this can lead to data breaches, especially in relation to data compliance regulations such as GDPR. An Internet Protocol (IP) address is an identifier that is assigned to a networked device to enable it to communicate over networks that use IP. Thus, in applications which are privacy-aware, we may aim to hide the IP address while aiming to determine if the address comes from a blacklist. One solution to this is to use homomorphic encryption to match an encrypted version of an IP address to a blacklisted network list. This matching allows us to encrypt the IP address and match it to an encrypted version of a blacklist. In this paper, we use the OpenFHE library [1] to encrypt network addresses with the BFV homomorphic encryption scheme. In order to assess the performance overhead of BFV, we implement a matching method using the OpenFHE library and compare it against partial homomorphic schemes, including Paillier, Damgard-Jurik, Okamoto-Uchiyama, Naccache-Stern and Benaloh. The main findings are that the BFV method compares favourably against the partial homomorphic methods in most cases.

1 Introduction

Data regulations such as GDPR demand greater control of Personally Identifiable Information (PII). In many areas of cybersecurity, we provide linkages between entities and their associated IP address and where revealing an IP address can often identify a person or organisation that is involved in an investigation. With this, we could define a blacklist of networks that we need to identify if specific source IP address is included. One of the best ways to preserve privacy is with the use of homomorphic encryption, where we can encrypt both the target IP address and the blacklist and match them without revealing any additional information.

Homomorphic encryption allows us to take the plaintexts m_1 and m_2 encrypt them using a secret key k , and perform operations such that:

$$\text{Enc}_k(m_1 \circ m_2) = \text{Enc}_k(m_1) \circ \text{Enc}_k(m_2)$$

where $\circ$ could potentially be any operator, such as add, multiply, logical and, or logical or. With symmetric key encryption, we use the same key to decrypt as we do to encrypt. Overall, in analysing the IP matching problem, we need to either conduct bitwise homomorphic encryption or use a homomorphic subtraction method.

In the past, partial homomorphic methods (PHE) have been used within privacy-aware methods for network analysis. This includes Tusa *et al.*, who used the Paillier method to implement privacy-aware routing [2]. These methods often have fairly good performance levels, but they do not implement a full range of mathematical operations and thus often fail to scale on a large-scale basis, especially where the full range of operations is required.

This paper thus provides a new method for the usage of fully homomorphic encryption to match IP addresses to a blacklist of network addresses without revealing the IP address or the blacklist.

2 Fully Homomomorphic Encryption

Homomorphic encryption is a method of encryption which supports operations over encrypted data. In 1978, Rivest, Adleman, and Dertouzos [3] were the first to explore the possibilities of using the natural homomorphic properties of the RSA public key encryption scheme. The RSA scheme only supports the evaluation of arithmetic multiplication over ciphertexts. RSA is an example of Partial Homomorphic Encryption (PHE), which is a scheme that supports the evaluation of only a single type of operation on ciphertexts. Fully Homomorphic Encryption (FHE) can support every operation. Since Gentry defined the first FHE method [4] in 2009, there have been four main generations of homomorphic encryption:

- 1st generation: Gentry’s method uses integers and lattices. [5] including the DGHV method.
- 2nd generation. Brakerski, Gentry and Vaikuntanathan’s (BGV) and Brakerski/ Fan-Vercauteren (BFV) use a Ring Learning With Errors approach [6]. The methods are similar to each other and have only a small difference between them.
- 3rd generation: These include DM (also known as FHEW) and CGGI (also known as TFHE) and support the integration of Boolean circuits for small integers.
- 4th generation: CKKS (Cheon, Kim, Kim, Song) and which uses floating-point numbers [7].

2.1 Public key or symmetric key

Homomorphic encryption can be implemented either with a symmetric key or an asymmetric (public) key. With symmetric key encryption, we use the same key to encrypt as we do to decrypt, whereas, with an asymmetric method, we

use a public key to encrypt and a private key to decrypt. In Figure 1, we use asymmetric encryption with a public key (pk) and a private key (sk). With this, Bob, Alice and Peggy will encrypt their data using the public key to produce ciphertext, and then we can operate on the ciphertext using arithmetic operations. The result can then be revealed by decrypting with the associated private key. We can also use symmetric key encryption, where the data is encrypted with a secret key, and which is then used to decrypt the data. In this case, the data processor (Trent) should not have access to the secret key, as they could decrypt the data from the providers.

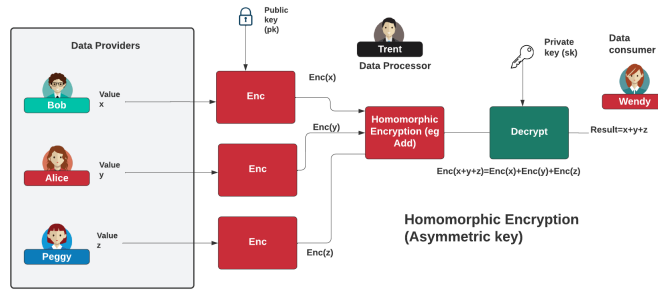


Fig. 1. Asymmetric encryption (public key)

2.2 BGV and BFV

With BGV and BFV [8], we use a Ring Learning With Errors (LWE) method [6]. With BGV, we define a modulus (q), which constrains the range of the polynomial coefficients. Overall, the methods use a moduli, which can be defined within different levels. We then initially define a finite group of $\mathbb{Z}_q$ and then make this a ring by dividing our operations with $(x^n + 1)$ and where $n - 1$ is the largest power of the coefficients. The message can then be represented in binary as:

$$m = a_{n-1}a_{n-2}...a_0 \quad (1)$$

This can be converted into a polynomial with:

$$\mathbf{m} = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + ... + a_1x + a_0 \pmod{q} \quad (2)$$

The coefficients of this polynomial will then be a vector. Note that for efficiency, we can also encode the message with ternary (such as with -1, 0 and 1). We then define the plaintext modulus with:

$$t = p^r \quad (3)$$

and where p is a prime number and r is a positive number. We can then define a ciphertext modulus of q , and which should be much larger than t . To encrypt with the private key of $\mathbf{s}$, we implement:

$$(c_0, c_1) = \left(\frac{q}{t} \cdot \mathbf{m} + \mathbf{a} \cdot \mathbf{s} + e, -\mathbf{a} \right) \mod q \quad (4)$$

To decrypt:

$$m = \left\lfloor \frac{t}{q} (c_0 + c_1) \cdot \mathbf{s} \right\rfloor \quad (5)$$

Noise and computation But each time we add or multiply, the error also increases. Thus, bootstrapping is required to reduce the noise. Overall, addition and plaintext/ciphertext multiplication is not a time-consuming task, but ciphertext/ciphertext multiplication is more computationally intensive. The most computational task is typically the bootstrapping process, and the ciphertext/-ciphertext multiplication process adds the most noise to the process.

Parameters We thus have a parameter of the ciphertext modulus (q) and the plaintext modulus (t). Both of these are typically to the power of 2. An example of q is 2^{240} and for t is 65,537. As the value of 2^q is likely to be a large number, we typically define it as a $\log_q$ value. Thus, a ciphertext modulus of 2^{240} will be 240 as defined as a $\log_q$ value.

3 Partial homomorphic encryption

With partial homomorphic encryption (PHE), we can implement some form of arithmetic operation in a homomorphic way. These methods include RSA, ElGamal, Paillier [9], Exponential ElGamal, Elliptic Curve ElGamal [10], Paillier [9], Damgard-Jurik [11], Okamoto-Uchiyama [12], Benaloh [13], Naccache-Stern [14], and Goldwasser-Micali [15]. Overall, we can use RSA and ElGamal for multiplicative homomorphic encryption; Paillier, Exponential ElGamal, Elliptic Curve ElGamal, Damgard-Jurik, Okamoto-Uchiyama, Benaloh and Naccache-Stern for additive homomorphic encryption; and Goldwasser-Micali for XOR homomorphic encryption.

3.1 Paillier

The Paillier cryptosystem [9] is a partial homomorphic encryption (PHE) method that can perform addition, subtraction, and scalar multiplication. Thus we get:

$$Enc_k(A + B) = Enc_k(A) + Enc_k(B) \quad (6)$$

$$Enc_k(A - B) = Enc_k(A) - Enc_k(B) \quad (7)$$

$$Enc_k(A \cdot B) = A \cdot Enc_k(B) \quad (8)$$

If we take two values: m_1 and m_2 , we get two encrypted values of $Enc(m_1)$ and $Enc(m_2)$. We can then multiply the two cipher values to get $Enc(m_1 + m_2)$. We can then decrypt to get $m_1 + m_2$. Along with this, we can also subtract to $Enc(m_1 - m_2)$. This is achieved by taking the inverse modulus of $Enc(m_2)$ and multiplying it with $Enc(m_1)$. Finally, we can perform a scalar multiply to get $Enc(m_1 \cdot m_2)$ and which is generated from $Enc(m_1)^{m_2}$.

First we select two large prime numbers (p and q) and compute:

$$N = pq \quad (9)$$

$$PHI = (p - 1)(q - 1) \quad (10)$$

$$\lambda = \text{lcm}(p - 1, q - 1) \quad (11)$$

and where lcm is the least common multiple. We then select a random integer g for:

$$g \in \mathbb{Z}_{nN^2}^* \quad (12)$$

We must make sure that n divides the order of g by checking the following:

$$\mu = (L(g^\lambda \pmod{n}^2))^{-1} \pmod{N} \quad (13)$$

and where L is defined as $L(x) = \frac{x-1}{N}$. The public key is (N, g) , and the private key is (λ, μ) .

To encrypt a message (M), we select a random r value and compute the ciphertext of:

$$c = g^m \cdot r^N \pmod{N^2} \quad (14)$$

and then to decrypt:

$$m = L(c^\lambda \pmod{N}^2) \cdot \mu \pmod{N} \quad (15)$$

For adding and scalar multiplying, we can take two ciphers (C_1 and C_2), and get:

$$C_1 = g^{m_1} \cdot r_1^N \pmod{N^2} \quad (16)$$

$$C_2 = g^{m_2} \cdot r_2^N \pmod{N^2} \quad (17)$$

If we now multiply them, we get:

$$C_1 \cdot C_2 = g^{m_1} \cdot r_1^N \cdot g^{m_2} \cdot r_2^N \pmod{N^2} \quad (18)$$

$$C_1 \cdot C_2 = g^{m_1+m_2} \cdot r_1^N \cdot r_2^N \pmod{N^2} \quad (19)$$

3.2 Benaloh

In 1986, Josh (Cohen) Benaloh published on *A Robust and Verifiable Cryptographically Secure Election Scheme* [13,16]. Within it, Josh outlined a public key encryption method and where Bob could generate a public and a private key. Alice could then use the public key to encrypt data for Bob, and then Bob could use the associated private key to decrypt it. It has the advantage of supporting additive homomorphic encryption. For the Benaloh method, to generate a key pair, Bob generates p and q , and which are two large prime numbers, which are two large distinct prime numbers. Next, he computes:

$$n = pq \quad (20)$$

$$\phi(n) = (p-1)(q-1) \quad (21)$$

Bob then selects a block size (r) so that:

- r divides $p - 1$
- $\gcd(r, (p - 1)/r) = 1$
- $\gcd(r, q - 1) = 1$

Next Bob selects y so that:

$$x = y^{\phi(n)/r} \pmod{n} \neq 1 \quad (22)$$

Bob's private key is (p, q) , and his public key is (y, r, n) . To encrypt data for Bob, Alice selects a message (m) and uses Bob's public key of (y, r, n) . First, she selects a random value of u and which is between 0 and n . Alice then encrypts with:

$$c = y^m u^r \pmod{n} \quad (23)$$

She sends this ciphertext to Bob. He will then decrypt with:

$$a = c^{\phi(n)/r} \pmod{n} \quad (24)$$

Bob lets $md = 0$. If $x^{md} \pmod{n} \neq a$ the Bob increments md by 1. He keeps doing this until:

$$x^{md} \pmod{n} = a \quad (25)$$

The value of md is then the original plaintext. One of the advantages of the Benaloh method is that we can perform additive homomorphic encryption. If we have two messages, we multiply the ciphers together for each message:

$$c_1 = y^{m_1} u^r \pmod{n} \quad (26)$$

$$c_2 = y^{m_2} u^r \pmod{n} \quad (27)$$

$$c = c_1 \cdot c_2 \quad (28)$$

This will give:

$$c = y^{m_1+m_2} u^r \pmod{n} \quad (29)$$

3.3 Okamoto-Uchiyama

With the Okamoto-Uchiyama method [12,17], we can perform additive and scalar multiply homomorphic encryption. A public/private key pair is generated as follows:

- Generate large primes p and q and set $n = p^2q$.
- $g \in (\mathbb{Z}/n\mathbb{Z})^*$ such that $g^{p-1} \not\equiv 1 \pmod{p^2}$.
- Let $h = g^n \pmod{n}$.

The public key is (n, g, h) , and then the private key is (p, q) . To encrypt a message m , where m is taken to be an element in 2^{k-1} . We then select $r \in \mathbb{Z}/n\mathbb{Z}$ at random. The cipher is then:

$$C = g^m h^r \pmod{n} \quad (30)$$

Next, we define the function of: $L(x) = \frac{x-1}{p}$.

We then decrypt with:

$$m = \frac{L(C^{p-1} \pmod{p^2})}{L(g^{p-1} \pmod{p^2})} \pmod{p} \quad (31)$$

3.4 Naccache-Stern

With the Naccache-Stern method [14,18], we select a large prime number (p) . We then select a value (n) and for i from 0 to n , we select the first n prime numbers $(p_0 \dots p_{n-1})$ of which p_0 is 2. We must make sure that:

$$\prod_{i=0}^n p_i \mid n; p \quad (32)$$

For our secret key (s) we make sure that:

$$\gcd(s, p-1) = 1 \quad (33)$$

To compute the public key (v_i) , we calculate: $v_i = \sqrt[s]{p_i} \pmod{p}$ To encrypt, we take a message of m and then determine the message bits of m_i . We can then cipher with the public key:

$$c = \prod_{i=0}^n v_i^{m_i} \pmod{p} \quad (34)$$

and then to decrypt:

$$m = \sum_{i=0}^n \frac{2^i}{p_i - 1} \times (\gcd(p_i, c^s \pmod{p}) - 1) \quad (35)$$

3.5 Goldwasser–Micali

With public key encryption, Alice could have two possible messages (a '0' or a '1') that she sends to Bob. If Eve knows the possible messages (a '0' or a '1'), she will then cipher each one with Bob's public key and then match the results against the cipher message that Alice sends. Eve can thus determine what Alice has sent to Bob. In order to overcome this, the Goldwasser–Micali (GM) method [19] is used as a public key algorithm that uses a probabilistic public-key encryption scheme. In this case, we will implement an XOR homomorphic encryption operation. For an input of 17 (10001) and 16 (10000), we will get a result of 00001 (1).

In a probabilist encryption method, Alice selects the plaintext (m) and a random string (r) . Next, she uses Bob's public key to encrypt the message pair of (m, r) . If the value is random, then Eve will not be able to use the range of messages and random values used. If Bob wants to create his public and private keys. He first selects two random prime numbers for his private key and then calculates N :

$$N = pq \quad (36)$$

The values of p and q will be his private key, and N will form part of his public key. For the second part of his public key, he determines:

$$a = \text{pseudosquare}(p, q) \quad (37)$$

For this, we determine if we can find, for a given value of a , which has no solutions:

$$u^2 \equiv a \pmod{p} \quad (38)$$

$$u^2 \equiv a \pmod{q} \quad (39)$$

This means that there are no quadratic residues. Bob's public key is (N, a) and the private key is (p, q) . The key encryption method becomes:

- Bob selects p and q .
- Bob selects a with $\left(\frac{a}{p}\right) = \left(\frac{a}{q}\right) = -1$. This is a Jacobi symbol calculation.
- Bob publishes N and a .

To encrypt for Bob:

- Select a bit to encrypt $m \in \{0, 1\}$.
- Alice uses Bob's values of N, a to compute:
- $c = r^2 \pmod{N}$ if $m = 0$
- $c = ar^2 \pmod{N}$ if $m = 1$

Alice chooses r at random, and thus, Eve will not be able to spot the message, as the random values will consist of all possible squares modulo N when $m = 0$. Alice sends c to Bob. To decrypt, Bob then computes $\left(\frac{c}{p}\right)$ and gets:

$$m = 0 : \text{if } \left(\frac{c}{p}\right) = 1 \quad (40)$$

$$m = 1 : \text{if } \left(\frac{c}{p}\right) = -1 \quad (41)$$

3.6 Damgard-Jurik

With the Damgard-Jurik method [11] we select two large prime numbers (p and q) and compute:

$$n = pq \quad (42)$$

$$\phi = (p-1)(q-1) \quad (43)$$

$$\lambda = \text{lcm } \phi \quad (44)$$

and where lcm is the least common multiple. We then select a random integer g for:

$$g \in \mathbb{Z}_{n^2}^* \quad (45)$$

We must make sure that n divides the order of g by checking the following:

$$\mu = (L(g^\lambda \pmod{n^2}))^{-1} \pmod{n} \quad (46)$$

and where L is defined as $L(x) = \frac{x-1}{n}$. The public key is (n, g) and the private key is (λ, μ) . To encrypt a message (M), we select a random r value and compute the ciphertext of:

$$c = g^m \cdot r^n \pmod{n^{s+1}} \quad (47)$$

and then to decrypt:

$$m = L(c^\lambda \pmod{n^2}) \cdot \mu \pmod{n}^s \quad (48)$$

If we take two ciphers (C_1 and C_2), we get:

$$C_1 = g^{m_1} \cdot r_1^n \pmod{n^2} \quad (49)$$

$$C_2 = g^{m_2} \cdot r_2^n \pmod{n^2} \quad (50)$$

If we now multiply them, we get:

$$C_1 \cdot C_2 = g^{m_1} \cdot r_1^n \cdot g^{m_2} \cdot r_2^n \pmod{n^2} \quad (51)$$

$$C_1 \cdot C_2 = g^{m_1+m_2} \cdot r_1^n \cdot r_2^n \pmod{n^2} \quad (52)$$

Adding two values requires the multiplication of the ciphers. If we now divide them, we get:

$$\frac{C_1}{C_2} = \frac{g^{m_1} \cdot r_1^n}{g^{m_2} \cdot r_2^n} \pmod{n^2} \quad \frac{C_1}{C_2} = g^{m_1-m_2} \frac{r_1^n}{r_2^n} \pmod{n^2}$$

Thus, subtraction is equivalent to a divide operation. For this, we perform a modular divide operation.

4 Methodology

An IPv4 address has four main fields that are defined with integer values in the range 0-255. An example is 12.23.45.67, which is a 32-bit address value and where each of the fields is identified with an 8-bit value. The address then splits into a network part and a host part, such as where 12.23.46.0 might identify a network address, and 0.0.0.67 will identify the host part. Overall, we define the network part with a subnet mask, and where bits that are set to a 1 identify the network part, and where we have a 0, we define the host part. We can then vary the number of 1's from 0 to 32. An example subnet mask where the network part is 24 bits long is 0xfffff00, and which can be represented by 255.255.255.0. This is often identified by the number of bits, such as 1's in the subnet mask, such as '/24'. An example IP address could thus be '192.168.0.10/24', and where the network part is '192.168.0.0' and the host part is '0.0.0.24'.

4.1 PHE Subtraction method

Algorithm 1 defines the method used for subtractive homomorphic encryption. In this, we convert the target IP address and the blacklisted network address to 32-bit integer values. The subnet mask then defines the network part to match. Again, this will be an integer value, and where the 1's identify the network part and the 0's will identify the host part. Overall, we are only interested in matching the network part of the target address to the blacklisted network address. We can then create a BFV keypair with a public key (pk) and a private key (sk). The homomorphic public key is then used to encrypt the target IP address and also the blacklisted address. Once encrypted, we can then perform a homomorphic subtraction. If the network part of the target IP address and the network address match, the result will be an encrypted value of zero. We can then decrypt the result of the homomorphic subtraction, and if we get a zero, we know that the IP address is contained in the blacklist.

Algorithm 1 FHE for IP detection

```
1: Set  $IP$  with the address to find for an integer
2: Set  $Network$  for the blacklisted addresses as an integer
3: Set  $Subnet_{mask}$  as the subnet of blacklist
4: Generate  $pk, sk$  for homomorphic key pair
5:  $IP_e = Enc(IP, pk)$ 
6:  $Blacklist = Network \wedge Subnet_{mask}$ 
7:  $Blacklist_e = Enc(Blacklist, pk)$ 
8:  $Enc_{diff} = IP_e - Blacklist_e$ 
9: if  $Enc_{diff} = 0$  then
10:   Address is in the blacklist
11: else
12:   Address is not in the blacklist
13: end if
```

4.2 Goldwasser–Micali XOR method

For the Goldwasser–Micali partial homomorphic encryption method, we can use the XOR operation, and where we can XOR the blacklist network address with the network address of the target IP address. This method is defined in Algorithm 2.

Algorithm 2 FHE for IP detection

```
1: Set  $IP$  with the address to find for an integer
2: Set  $Network$  for the blacklisted addresses as an integer
3: Set  $Subnet_{mask}$  as the subnet of blacklist
4: Generate  $pk, sk$  for homomorphic key pair
5:  $IP_e = Enc(IP, pk)$ 
6:  $Blacklist = Network \wedge Subnet_{mask}$ 
7:  $Blacklist_e = Enc(Blacklist, pk)$ 
8:  $Enc_{diff} = IP_e \oplus Blacklist_e$ 
9: if  $Enc_{diff} = 0$  then
10:   Address is in the blacklist
11: else
12:   Address is not in the blacklist
13: end if
```

4.3 OpenFHE parameters

The parameters that need to be set within OpenFHE for BFV are:

- *Scheme*. This defines the scheme to be used. In the case of BFV, this is set to BFVRNS_SCHEME.
- *RDim*. This defines the size of the lattice ring dimension. A typical value for this is 16,384.

- *MultDepth*. This is the multiplication depth and is the maximum number of sequential (cascaded) multiplications that are performed on encrypted data before decryption fails due to excessive noise accumulation.
- *PtMod*. This defines the plaintext modulus, and it needs to be a prime number that is larger than the number of bits in the plaintext.

The parameter set needs to support integer values up to 32 bits. A recommended setup for 41-bit resolution for the plaintext (*PtMod*) is set at 35,184,372, 744,193, along with the other parameters defined at [20]. The FHE and PHE code is given at [21].

Table 1: Results for homomorphic operations for IP matching

Method	Key pair (ms)	Encrypt (ms)	Operation and decrypt (ms)
BFV (OpenFHE) [1]	93.1	270.2	16.4
Paillier [9]	55.7	168.6	66.3
Damgard-Jurik [11]	83.6	443.5	155.2
Okamoto-Uchiyama [12]	128.9	206.9	18.0
Naccache-Stern [14]	48.2	0.2	0.6
Benaloh	6.0	0.3	0.2
Goldwasser-Micali [15]	0.3	0.5	1.6

5 Evaluation

The coding for fully homomorphic encryption using OpenFHE is defined in the Coding section. The results for a t3.medium instance on AWS are given in Table 1, which includes a comparison with partial homomorphic methods using the PHE Library [22]. The time to set up the key pair and the context for the encryption is measured at an average of 93 ms. The greatest overhead is then the time it takes to encrypt the values, which has an average time of around 270 ms. The subtraction and decryption timing then comes in around 16 ms. It can be seen that the encryption process provides the largest overhead in IP address matching. We can see that the Benaloh and Goldwasser-Micali methods are by far the fastest. The BFV method has comparable performance to the PHE methods and is actually faster in the homomorphic encryption operation than Paillier, Damgard-Jurik, and Okamoto-Uchiyama. The Naccache-Stern method also performs well, especially in the encryption and decryption process. Overall, the encryption process tends to have the greatest processing overhead.

The results in Table 2 outline the time to encrypt a number of IP addresses into a data store, and then the total time to search the whole of the data store and find the last matched IP address. As we can see, the encryption time is the most considerable processing overhead.

Table 2: Results for fully homomorphic encryption using OpenFHE and matching for a range of random addresses

IP addresses	Time to encrypt (secs)	Max time to search (sec)
50	6.47	1.08
100	11.55	2.48
200	23.25	4.09
400	125.55	8.13
800	238.93	16.57

6 Conclusion

The increasing requirements for privacy-aware cybersecurity provide opportunities to encrypt data using homomorphic encryption. This paper outlines a method that requires plaintext to support 32 bits and requires a larger plaintext modulus than is used by default in applications. The paper has thus outlined a method that uses the popular OpenFHE library and has fairly reasonable overheads in latency in creating the homomorphic encryption keys and in encrypting, processing, and decrypting data.

References

1. W. J. Buchanan, “Homomorphic encryption (openfhe),” <https://asecuritysite.com/openfhe>, Asecuritysite.com, 2025, accessed: February 20, 2025. [Online]. Available: <https://asecuritysite.com/openfhe>
2. F. Tusa, D. Griffin, and M. Rio, “Homomorphic routing: Private data forwarding in the internet,” in *Proceedings of the 2nd ACM SIGCOMM Workshop on Future of Internet Routing & Addressing*, 2023, pp. 1–7.
3. R. L. Rivest, L. Adleman, M. L. Dertouzos *et al.*, “On data banks and privacy homomorphisms,” *Foundations of secure computation*, vol. 4, no. 11, pp. 169–180, 1978.
4. C. Gentry, “A fully homomorphic encryption scheme,” 2009, crypto.stanford.edu/craig.
5. M. Van Dijk, C. Gentry, S. Halevi, and V. Vaikuntanathan, “Fully homomorphic encryption over the integers,” in *Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*. Springer, 2010, pp. 24–43.
6. Z. Brakerski and V. Vaikuntanathan, “Efficient fully homomorphic encryption from (standard) lwe,” *SIAM Journal on computing*, vol. 43, no. 2, pp. 831–871, 2014.
7. J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I 23*. Springer, 2017, pp. 409–437.
8. W. J. Buchanan, “Homomorphic encryption and lattices,” <https://asecuritysite.com/lattice/bfv.bgv>, Asecuritysite.com, 2025, accessed: March 05, 2025. [Online]. Available: <https://asecuritysite.com/lattice/bfv.bgv>

9. P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *International conference on the theory and applications of cryptographic techniques*. Springer, 1999, pp. 223–238.
10. T. ElGamal, "A public key cryptosystem and a signature scheme based on discrete logarithms," *IEEE transactions on information theory*, vol. 31, no. 4, pp. 469–472, 1985.
11. I. Damgård, M. Jurik, and J. B. Nielsen, "A generalization of paillier's public-key system with applications to electronic voting," *International Journal of Information Security*, vol. 9, pp. 371–385, 2010.
12. T. Okamoto and S. Uchiyama, "A new public-key cryptosystem as secure as factoring," in *Advances in Cryptology—EUROCRYPT'98: International Conference on the Theory and Application of Cryptographic Techniques Espoo, Finland, May 31–June 4, 1998 Proceedings 17*. Springer, 1998, pp. 308–318.
13. J. D. Cohen and M. J. Fischer, *A robust and verifiable cryptographically secure election scheme*. Yale University. Department of Computer Science, 1985.
14. D. Naccache and J. Stern, "A new public-key cryptosystem," in *Advances in Cryptology—EUROCRYPT'97: International Conference on the Theory and Application of Cryptographic Techniques Konstanz, Germany, May 11–15, 1997 Proceedings 16*. Springer, 1997, pp. 27–36.
15. S. Goldwasser and S. Micali, "Probabilistic encryption & how to play mental poker keeping secret all partial information," in *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*, 2019, pp. 173–201.
16. W. J. Buchanan, "Benaloh public key encryption," <https://asecuritysite.com/homomorphic-partial/benaloh2>, Asecuritysite.com, 2025, accessed: February 21, 2025. [Online]. Available: <https://asecuritysite.com/homomorphic-partial/benaloh2>
17. —, "Okamoto uchiyama public key encryption," https://asecuritysite.com/homomorphic-partial/ou_homomorphic, Asecuritysite.com, 2025, accessed: February 21, 2025. [Online]. Available: https://asecuritysite.com/homomorphic-partial/ou_homomorphic
18. —, "Naccache stern public key encryption," https://asecuritysite.com/homomorphic-partial/nacc_enc, Asecuritysite.com, 2025, accessed: February 21, 2025. [Online]. Available: https://asecuritysite.com/homomorphic-partial/nacc_enc
19. O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game or a completeness theorem for protocols with honest majority," in *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, A. Aho, Ed. ACM Press, 1987, pp. 218–229.
20. OpenFHE, "Openfhe unit tests," <https://github.com/openfheorg/openfhe-development/blob/main/src/pke/unittest/UnitTestSHE.cpp>, OpenFHE.com, 2025, accessed: September 06, 2024. [Online]. Available: <https://github.com/openfheorg/openfhe-development/blob/main/src/pke/unittest/UnitTestSHE.cpp>
21. —, "Homomorphic processing of ip addresses," https://github.com/openfheorg/education/tree/main/examples/ip_masking, Asecuritysite.com, 2025, accessed: March 05, 2025. [Online]. Available: https://github.com/openfheorg/education/tree/main/examples/ip_masking
22. S. I. Serengil, "Phe library," <https://github.com/serengil/LightPHE>, PHE, 2025, accessed: February 21, 2025. [Online]. Available: <https://github.com/serengil/LightPHE>